

DQN & DDQN for Automated Stock Trading

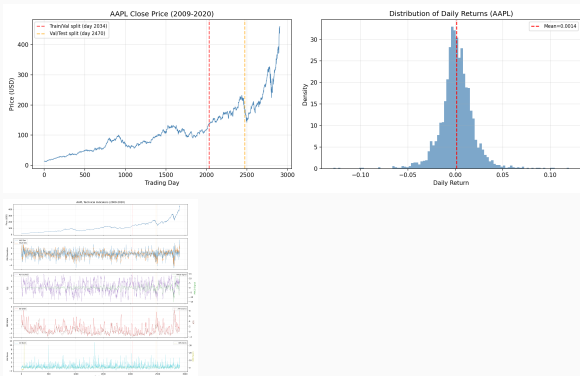
Implementation, Iterative Improvements & Results

Yufan Zhu Rutwik Shrikant Dharanappagoudar

April 26, 2026

Yufan: Data, Environment & Initial System

Yufan — Data Pipeline & Feature Engineering



AAPL is split chronologically into train, validation and test periods; all normalisation uses train-set statistics only.

Reproducible market-data pipeline

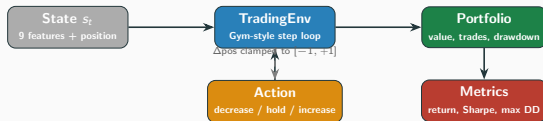
- Kaggle alincijov/trading loaded via kagglehub.
- DJIA ticker subset with AAPL case study.
- Chronological 70/15/15 train-val-test split.
- Raw returns preserved for rewards and portfolios.

Nine engineered indicators

Group	Features
Trend	Return, MA5 dev., MA20 dev.
Momentum	RSI-14, MACD
Volatility	Bollinger width, ATR
Volume	Normalised volume, OBV

Goal: give each agent the same clean, leakage-safe state representation before comparing algorithms.

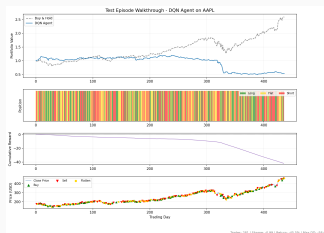
Yufan — Trading Environment & Evaluation



$$V_t = V_{t-1} (1 + \text{pos}_t \tilde{r}_t - c_t 1[\Delta \text{pos} \neq 0])$$
$$r_t = \log(V_t / V_{t-1}) + \lambda \min(0, \text{drawdown}_t)$$

Environment design choices

- Positions: short, flat, long.
- Actions move position by one step.
- Transaction cost: 10 bps per position change.
- Drawdown penalty discourages fragile portfolios.
- Full portfolio history tracked every episode.



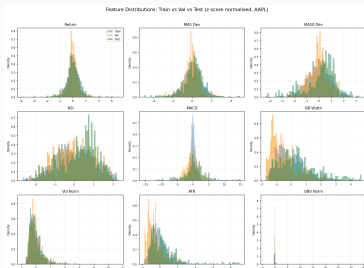
The walkthrough view links actions to portfolio value, cumulative reward and buy/sell timing.

Yufan — Initial Agents, Baselines & Report Assets

Initial from-scratch system

- Built the first complete codebase.
- Implemented QNetwork, replay buffer and target network.
- Added DQN/DDQN training and evaluation loops.
- Added Random Forest baseline with 200 trees.
- Produced data/training figures and appendix material.

Layer	Yufan output
Data	Kaggle download, indicators, splits
Env	State, reward, costs, metrics
Agents	First DQN/DDQN implementation
Baselines	Random Forest comparison
Report	Figures, appendix, writing



Why this section matters

The project depends on a consistent experimental scaffold: same features, same split, same portfolio accounting and same metrics for every model. This makes later DQN/DDQN refinements comparable instead of changing the task underneath the agent.

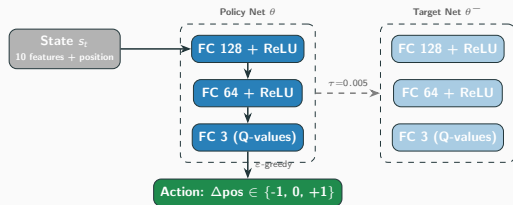
Presentation hand-off

Yufan establishes the dataset, environment and initial learning system; Rutwik then focuses on algorithmic refinements, stronger baselines and final result analysis.

Rutwik: DQN/DDQN

Refinements & Results

DQN & DDQN — Architecture & Implementation



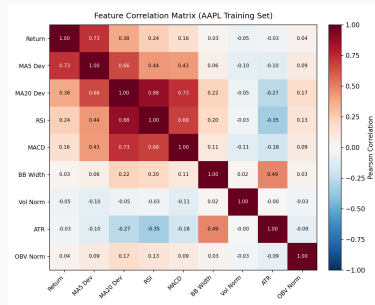
DDQN: select $a^* = \arg \max Q_\theta$, evaluate $Q_{\theta^-}(s', a^*)$

DQN: Replay buffer breaks temporal correlations; target network provides stable TD targets. Counterfactual replay improves sample efficiency.

DDQN: Decouples action *selection* (policy net) from *evaluation* (target net) to reduce overestimation bias.

10 Technical Features (z-score normalised):

Feature	Signal Type
Return, Dev5, Dev20	Trend / mean-reversion
Mom20	Momentum
RSI, MACD	Oscillator / crossover
BB Width, ATR	Volatility
Vol Norm, OBV Norm	Volume flow



Volume/volatility indicators are near-uncorrelated with trend indicators, suggesting reduced redundancy and more diverse signals.

Iterative Improvements Across Versions

Component	V1 (Initial)	V2 (Core)	V4/HEAD
Actions	Set position	Delta-based	Delta-based
Reward	Raw PnL	Log-return	Log-return
ϵ	Linear, 1.0 $\rightarrow$.05	Exp decay	0.2 $\rightarrow$.02
Target upd.	Hard (N=100)	Soft $\tau=.005$	Soft
Loss	MSE	Huber	Huber
LR / Batch	1e-3 / 64	5e-4 / 128	1e-4 / 128
Buffer	10 k	50 k	50 k
Episodes	Full dataset	252d random	252d random
Warm start	—	—	Momentum
Replay	Standard	Standard	+Counterfact.

Green = change introduced in that version.

Three most impactful changes:

1. Reward: Raw PnL $\rightarrow$ Log-Return

$$r_t = \log\left(\frac{V_{t+1}}{V_t}\right) \quad \text{Scale-invariant, additive}$$

Raw PnL was dominated by large price moves; log-return treats gains/losses symmetrically

and is additive over time, improving numerical stability for sequential optimisation.

2. Supervised Warm Start from Momentum Teacher

Pre-train Q-net via cross-entropy on momentum signal (20d lookback). Builds (s, a) pairs for

every reachable position so policy learns entry & exit. Lower $\epsilon_0=0.2$ preserves pre-trained weights.

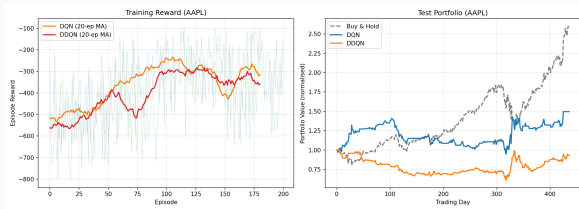
3. TDQN-Style Counterfactual Experience Replay

For each step, compute exact rewards for the 2 actions **not** taken. Pushes $3\times$ transitions per

step $\rightarrow$ significantly improves sample efficiency in practice. Possible in our deterministic

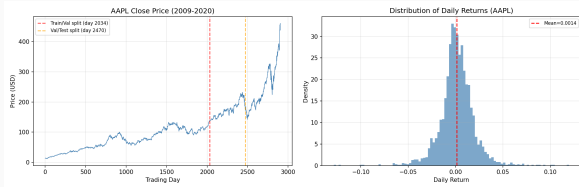
backtesting environment, where rewards depend only on price data.

Results & Analysis (AAPL, 2018–2020 Test Set)



Left: Training reward (20-ep MA). Agents generally trend upward. Right: Test portfolio value.

Modern RL agents (PPO, A2C, DDQN) consistently outpace Buy & Hold.



AAPL 2009–2020 with train/val/test splits. Test period is a strong bull market.

Test Performance:

Model	Return	Sharpe	Max DD
PPO	+184.3%	2.81	−16.7%
A2C	+182.9%	2.79	−17.1%
DDQN	+172.5%	2.73	−15.9%
DQN	+171.7%	2.73	−15.5%
Momentum	+167.0%	2.62	−17.5%
Buy & Hold	+159.3%	1.65	−31.4%
ARIMA	+121.1%	1.67	−18.8%
Random Forest	+81.1%	1.25	−26.8%

Key Takeaways:

- ✓ **RL outperforms baselines:** All RL agents beat Buy & Hold despite strong bull market constraints.
- ✓ **Actor-Critic superiority:** PPO and A2C yield top returns, better managing noisy financial sequences.
- ✓ **DDQN $\gtrsim$ DQN:** DDQN slightly outperforms DQN, consistent with reduced overestimation bias.
- ! RL models exhibit lower max drawdowns ($\sim 16\%$), roughly half that of B&H.